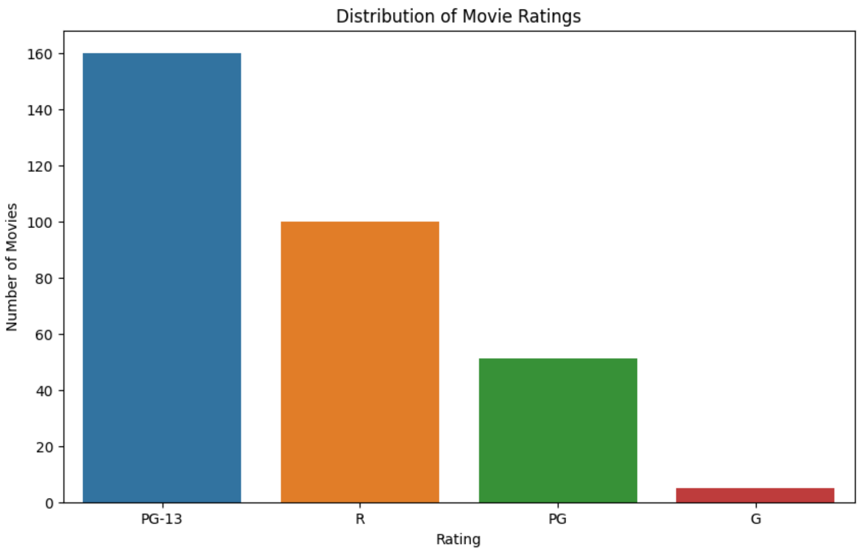
🎬 Comprehensive Movie Ratings Analysis Pipeline

Based on Rotten Tomatoes dataset (2010 and later, with audience count > 100,000).

In [1]:
```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

# Load the dataset
movies = pd.read_csv('Rotten Tomatoes Movies.csv')

# Filter columns and clean dates
movies = movies[['movie_title', 'rating', 'genre', 'in_theaters_date', 'runtime_in_minutes',
                 'tomatometer_rating', 'tomatometer_count', 'audience_rating', 'audience_count']]
movies['in_theaters_date'] = pd.to_datetime(movies['in_theaters_date'], errors='coerce')

# Filter for movies from 2010 onwards and with audience count > 100,000
movies = movies[movies['in_theaters_date'].dt.year >= 2010]
movies_popular = movies[movies['audience_count'] > 100000].copy()

# Feature engineering for genre flags
movies_popular['Animation'] = movies_popular.genre.str.contains('Animation', na=False).astype(int)
movies_popular['Action & Adventure'] = movies_popular.genre.str.contains('Action & Adventure', na=False).astype(int)
movies_popular['Comedy'] = movies_popular.genre.str.contains('Comedy', na=False).astype(int)

# Derived fields
movies_popular['year'] = movies_popular['in_theaters_date'].dt.year
```

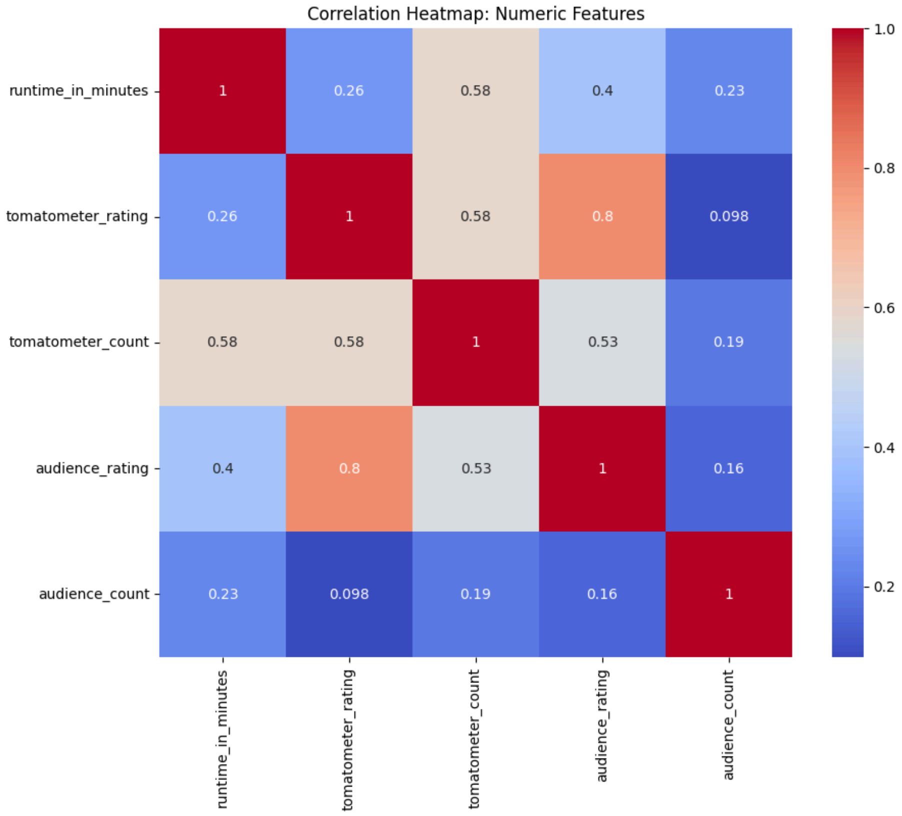
🔢 Distribution of Movie Ratings

In [2]:
```
plt.figure(figsize=(10, 6))
sns.countplot(data=movies_popular, x='rating', order=movies_popular['rating'].value_counts().index)
plt.title('Distribution of Movie Ratings')
plt.ylabel('Number of Movies')
plt.xlabel('Rating')
plt.show()
```



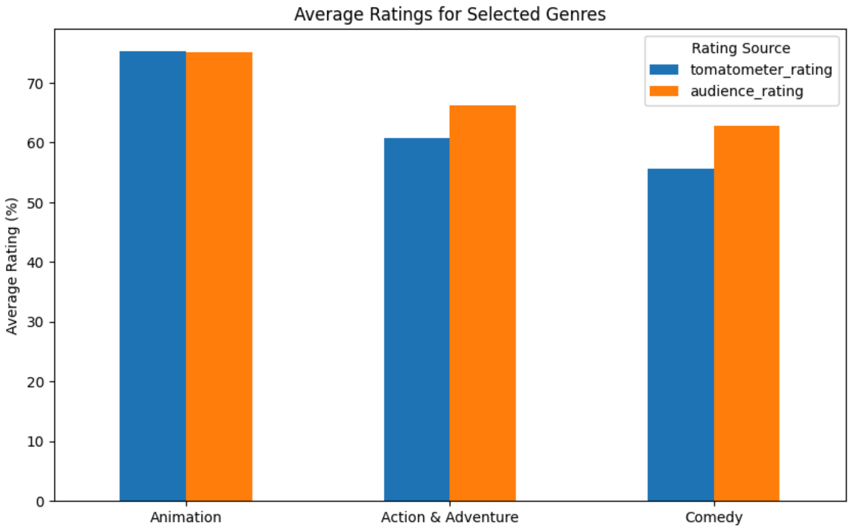
📈 Correlation Heatmap

In [3]:
```
plt.figure(figsize=(10, 8))
corr_matrix = movies_popular[['runtime_in_minutes', 'tomatometer_rating', 'tomatometer_count',
                              'audience_rating', 'audience_count']].corr()
sns.heatmap(corr_matrix, annot=True, cmap='coolwarm')
plt.title("Correlation Heatmap: Numeric Features")
plt.show()
```



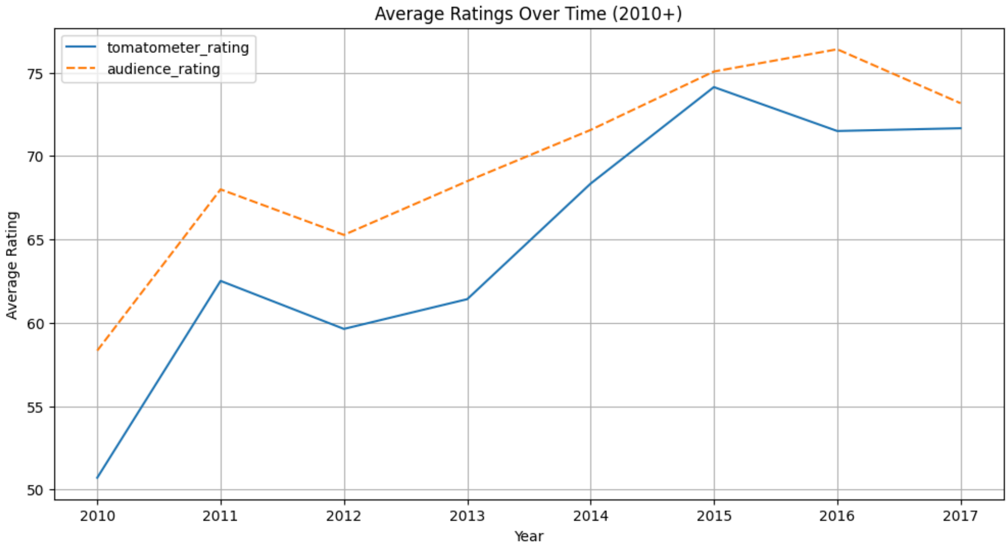
🎭 Genre vs Ratings

In [4]:
```python
avg_by_genre = pd.DataFrame({
    'Animation': movies_popular.groupby('Animation')[['tomatometer_rating', 'audience_rating']].mean().iloc[1],
    'Action & Adventure': movies_popular.groupby('Action & Adventure')[['tomatometer_rating', 'audience_rating']].mean().iloc[1],
    'Comedy': movies_popular.groupby('Comedy')[['tomatometer_rating', 'audience_rating']].mean().iloc[1]
}).T

avg_by_genre.plot(kind='bar', figsize=(10, 6))
plt.title("Average Ratings for Selected Genres")
plt.ylabel("Average Rating (%)")
plt.xticks(rotation=0)
plt.legend(title="Rating Source")
plt.show()
```



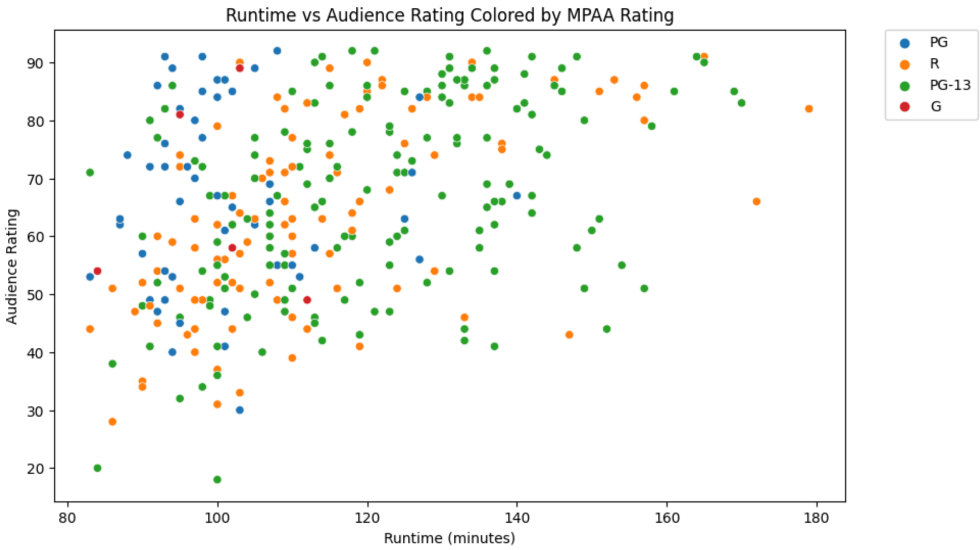
📅 Trends Over Time

In [5]:
```python
avg_rating_by_year = movies_popular.groupby('year')[['tomatometer_rating', 'audience_rating']].mean()

plt.figure(figsize=(12, 6))
sns.lineplot(data=avg_rating_by_year)
plt.title("Average Ratings Over Time (2010+)")
plt.xlabel("Year")
plt.ylabel("Average Rating")
plt.grid(True)
plt.show()
```



⏱️ Runtime vs Audience Rating

In [6]:
```python
plt.figure(figsize=(10, 6))
sns.scatterplot(data=movies_popular, x='runtime_in_minutes', y='audience_rating', hue='rating')
plt.title("Runtime vs Audience Rating Colored by MPAA Rating")
plt.xlabel("Runtime (minutes)")
plt.ylabel("Audience Rating")
plt.legend(bbox_to_anchor=(1.05, 1), loc=2, borderaxespad=0.)
plt.show()
```



🌟 Top 10 Most Reviewed Movies

In [7]:
```python
top_audience_count = movies_popular.nlargest(10, 'audience_count')[['movie_title', 'audience_count']]
plt.figure(figsize=(12, 6))
sns.barplot(data=top_audience_count, x='audience_count', y='movie_title', palette='viridis')
plt.title("Top 10 Most Reviewed Movies by Audience")
plt.xlabel("Audience Review Count")
plt.ylabel("Movie Title")
plt.show()
```